

한국식 야구기록지 데이터화 파이프라인

PyCharm + Claude Code 단계별 진행 가이드

Stage 1~4를 한꺼번에 만들지 않고, **한 단계씩 눈으로 확인하며** 쌓아 올리는 5일 로드맵입니다. 각 단계마다 Claude Code에 그대로 붙여 넣을 프롬프트와, 다음으로 넘어가도 되는지 판단하는 통과 기준을 함께 적었습니다.

0. 시작 전에 — 이 순서를 지켜야 하는 이유

가장 흔한 실패는 Stage 1~4를 한 번에 만들고 통째로 돌려보는 것입니다. 이러면 결과가 틀렸을 때 **정합이 어긋난 것인지, 크롭이 밀린 것인지, VLM이 잘못 읽은 것인지 구분할 수 없습니다**. 특히 크롭이 몇 픽셀 밀린 상태로 Stage 3을 돌리면 VLM은 헛것을 자신 있게 읽어내고, 그걸 알아챌 방법이 없습니다.

그래서 각 단계는 **사람 눈으로 확인할 수 있는 산출물**을 하나씩 남깁니다. 오버레이 이미지, 크롭 PNG, 판독 JSON, 위반 리포트 순서입니다.

일차	작업	눈으로 확인할 산출물	통과 기준
1일차	프로젝트 셋업 + 템플릿 좌표	template_overlay.png	모든 셀 경계선이 격자와 일치
2일차	Stage 1~2 (정합 + 크롭)	셀 PNG 108장	다이아몬드가 중앙에, 잘림 없음
3일차	골든셋 구축	golden/*.json	다이아몬드 30~50칸 정답 완비
4일차	Stage 3 (VLM 판독)	reading.json + 정확도	셀 정확도 측정값 확보
5일차	Stage 4 (규칙 검증)	verify.json	위반 패턴 3~5종 파악

3일차를 건너뛰고 싶어질 겁니다

골든셋 없이 Stage 3을 돌리면 '잘 되는 것 같다'는 감으로만 개발하게 됩니다. 프롬프트를 고쳤을 때 좋아졌는지 나빠졌는지 판단할 근거가 사라지고, 모델을 바꿔볼 수도 없습니다. 다이아몬드 30칸이면 충분히 반드시 만드세요.

1일차 — 프로젝트 셋업과 템플릿 좌표

1-1. PyCharm 프로젝트 생성

New Project → Pure Python → 인터프리터는 **Virtualenv, Python 3.11 이상**을 선택합니다. 위치는 경로에 한글이나 공백이 없는 곳으로 잡으세요 (OpenCV 경로 처리에서 문제가 생길 수 있습니다).

```
scoresheet/
├─ .venv/
├─ CLAUDE.md          ← Claude Code가 매번 읽는 프로젝트 규칙
├─ requirements.txt
├─ schema.py cache.py pipeline.py make_template.py
├─ stage1_align.py stage2_slice.py stage3_read.py stage4_verify.py
├─ template/          ← 빈 양식 스캔본 + 좌표 JSON
├─ sheets/            ← 판독할 기록지 스캔본
├─ work/              ← 중간 산출물 (git 제외)
└─ tests/golden/      ← 정답 데이터
```

1-2. Claude Code 설치와 PyCharm 연동

```
# 터미널 (PyCharm 하단 Terminal 탭에서 실행)
npm install -g @anthropic-ai/claude-code
claude          # 최초 1회 로그인

# PyCharm 플러그인
# Settings → Plugins → Marketplace → "Claude Code" 설치 후 재시작
```

플러그인을 설치하면 에디터에서 **Alt+Ctrl+K**로 Claude Code 창을 열 수 있고, 열려 있는 파일과 선택 영역이 자동으로 맥락에 포함됩니다. 터미널에서 claude로 직접 쓰는 것과 기능은 같습니다.

1-3. CLAUDE.md 작성 — 가장 중요한 파일

Claude Code는 매 세션 시작 시 프로젝트 루트의 CLAUDE.md를 자동으로 읽습니다. 여기에 설계 원칙을 적어두면 **매번 같은 설명을 반복하지 않아도 되고**, 세션이 길어져도 규칙에서 벗어나지 않습니다. 이 파일 하나가 작업 품질을 크게 좌우합니다.

```
# 한국식 야구기록지 OCR 파이프라인

## 무엇을 만드는가
수기 야구기록지 스캔본 → 구조화 데이터. 4단계 결정론적 파이프라인.

## 반드시 지킬 규칙
- 각 stage는 "입력 파일 → 출력 파일"인 순수 함수다. 인메모리 전달 금지.
- 모든 stage 간 데이터는 schema.py의 Pydantic 모델을 따른다.
- VLM은 자유 텍스트가 아니라 JSON 스키마로만 응답받고 Pydantic으로 검증한다.
- VLM 호출은 반드시 cache.py를 경유한다. 캐시 키는
  sha256(셀 이미지) + 모델ID + 프롬프트 버전.
- 프롬프트를 수정하면 stage3_read.py의 PROMPT_VERSION을 반드시 올린다.
- 에이전트 패턴은 쓰지 않는다. 예외는 stage4 재판독 루프뿐이며 아직 미구현.

## 코딩 스타일
- Python 3.11+, from __future__ import annotations, 타입힌트 필수
- 주석은 "무엇을"이 아니라 "왜"를 적는다
- 새 기능마다 tests/에 pytest 케이스 추가

## 작업 시 주의
- 골든셋(tests/golden/)은 정답 데이터다. 절대 수정하지 마라.
- work/ 아래 파일은 언제든 지워도 되는 중간 산출물이다.
- 나에게 확인 없이 API 키를 코드에 하드코딩하지 마라.
```

1-4. 템플릿 좌표 맞추기

빈 양식을 **300dpi 이상**으로 스캔해 template/ksbl_v1.blank.png로 저장한 뒤, 격자선을 검출해 좌표 힌트를 뽑습니다.

```
python make_template.py probe --image template/ksbl_v1.blank.png
# 출력된 세로선/가로선 좌표를 보고 make_template.py 상단 상수를 채운다
python make_template.py build
python make_template.py verify --image template/ksbl_v1.blank.png
```

마지막 verify가 만든 template_overlay.png를 PyCharm에서 열어 **확대해서 눈으로 확인하세요**. 빨간 사각형이 다이아몬드 격자와 정확히 겹쳐야 합니다. 여기서 어긋나면 이후 모든 단계가 조용히 망가집니다.

Claude Code에 줄 프롬프트 예시

```
make_template.py의 probe 출력이 아래와 같다.  
세로선 14개: [178, 466, 684, 902, 1120, ...]  
가로선 11개: [298, 450, 602, 754, ...]  
첨부한 오버레이 이미지를 보면 다이아몬드 셀이 오른쪽으로 조금씩  
밀려 있다. 상단 상수를 어떻게 조정해야 하는지 계산해서 수정해줘.
```

2일차 — Stage 1~2 실행과 크롭 검수

정합과 크롭까지만 돌립니다. VLM은 아직 부르지 않습니다. 돈이 들지 않는 단계에서 품질을 확보하는 것이 목적입니다.

```
python pipeline.py run --scan sheets/game001.png --sheet-id game001 \  
    --from-stage read --force      # read 이후는 아직 안 돌린다는 뜻이 아니라  
                                   # 실제로는 stage 2까지만 확인하고 중단  
  
# 더 간단하게: 파이썬 콘솔에서 stage 1~2만 직접 호출
```

확인해야 할 것

- 정합 점수가 0.80 이상인가 (work/game001/.align.done 확인)
- work/game001/slice/game001/의 셀 PNG를 20~30장 열어볼 것
- 다이아몬드가 셀 중앙에 있고, 위아래 칸의 필기가 섞여 들어오지 않았는가
- 빈 셀 판정이 맞는가 — 기록이 있는데 is_blank=true면 임계값을 낮춰야 함
- .nogrid.png에서 격자만 지워지고 필기는 남아 있는가

빈 셀 오판이 가장 위험합니다

기록이 있는 칸을 빈 칸으로 판정하면 VLM 호출 자체가 생략되어, 그 타석은 조용히 누락됩니다. 오류 메시지도 없습니다. INK_THRESHOLD를 조금씩 낮춰가며 오탐이 0이 되는 값을 찾으세요. 빈 칸을 실수로 호출하는 건 돈만 조금 더 들 뿐 안전합니다.

Claude Code에 줄 프롬프트 예시

```
work/game001/slice/ 에 크롭된 셀들을 확인했는데 두 가지 문제가 있다.  
1. 다이아몬드 셀 아래쪽이 3~5px 잘려서 로마숫자가 일부 안 보인다  
2. 좌측 투구 마크열이 옆 칸까지 침범해서 같이 잘려 들어온다  
  
stage2_slice.py의 CROP_PAD와 make_template.py의 CELL_H를  
어떻게 조정하면 좋을지 제안하고, 수정 후 다시 크롭해서  
셀 5개를 샘플로 저장하는 스크립트도 만들어줘.
```

3일차 — 골든셋 구축

다이아몬드 셀 **30~50칸**에 손으로 정답을 답니다. 반나절이면 끝나고, 이후 모든 개선의 판단 기준이 됩니다. 실제 경기 결과를 알고 있는 시트를 쓰면 정답 달기가 훨씬 쉽습니다.

```
# tests/golden/game001.json
{
  "d.r01.i01": {"reached": "1B", "result_code": "B", "scored": false},
  "d.r02.i01": {"reached": "none", "result_code": "6-4", "out_order": 1},
  "d.r05.i03": {"reached": "HOME", "scored": true, "rbi": 2}
}
```

Claude Code에 줄 프롬프트 예시

tests/golden/game001.json 을 정답으로 삼아, work/.../reading.json 과 비교해서 셀 단위 정확도를 내는 pytest를 만들어줘.

요구사항:

- 필드별 정확도를 따로 낼 것 (reached / result_code / out_order / scored)
- 틀린 셀은 셀ID, 정답, 판독값, 크롭 이미지 경로를 표로 출력
- pytest -q 로 돌리면 요약만, -v 면 틀린 셀 전체가 보이게
- 골든셋 파일은 절대 수정하지 말 것

4일차 — Stage 3 VLM 판독

이제 처음으로 돈이 나갑니다. **반드시 한 장으로만** 시작하세요. 캐시가 있으므로 재실행해도 중복 과금은 없습니다.

```
# .env 또는 PyCharm Run Configuration의 환경변수에 키 설정
# GEMINI_API_KEY=... (코드에 하드코딩 금지)

python pipeline.py run --sheet-id game001 --from-stage read
pytest tests/test_accuracy.py -v # 골든셋 대비 정확도 확인
```

프롬프트 개선 사이클

1. 정확도를 측정하고, 어떤 필드가 가장 많이 틀리는지 본다
2. 틀린 셀의 크롭 이미지를 직접 열어본다 — 사람이 봐도 애매하면 프롬프트 문제가 아니다
3. DIAMOND_SYSTEM 문구를 수정한다
4. **PROMPT_VERSION**을 올린다 (v4 → v5). 캐시가 자동 무효화된다
5. 재실행 후 정확도를 다시 측정해 개선 여부를 확인한다

PROMPT_VERSION 올리는 것을 잊지 마세요

이걸 빼먹으면 캐시가 옛 결과를 그대로 돌려주고, 프롬프트를 아무리 고쳐도 정확도가 미동도 하지 않습니다. 원인을 찾는 데 몇 시간을 태우기 딱 좋은 함정입니다.

Claude Code에 줄 프롬프트 예시

골든셋 대비 정확도가 이렇게 나왔다.
reached 0.91 / result_code 0.78 / out_order 0.72 / scored 0.96

out_order(로마숫자 I,II,III)가 가장 나쁘다. 틀린 셀을 보니 II와 III을 혼동하는 경우가 대부분이다.

DIAMOND_SYSTEM 프롬프트에서 로마숫자 판독 지침을 강화하는 안을 2가지 만들어주고, PROMPT_VERSION을 각각 v5a/v5b로 두고 두 버전을 같은 셀에 돌려 비교하는 스크립트를 작성해줘.

5일차 — Stage 4 규칙 검증

--auto-repair는 켜지 마세요. 먼저 위반이 실제로 어떤 모습으로 나오는지 관찰하는 것이 목적입니다. 보정이 틀리면 틀린 줄도 모르고 데이터가 오염됩니다.

```
python pipeline.py run --sheet-id game001 --from-stage verify
python pipeline.py review --sheet-id game001
```

위반 목록을 보면 몇 가지 패턴으로 뭉칩니다. 그 패턴을 보고 규칙을 하나씩 추가하세요. 상상으로 규칙 20개를 미리 짜면 절반은 안 걸리고, 정작 필요한 건 빠져 있습니다. 규칙 하나는 10줄짜리 순수 함수이므로 나중에 추가해도 부담이 없습니다.

추가할 만한 규칙 후보

- 잔루 정합 — 이닝 종료 시 루상 주자 수 = 집계란 잔루
- 타순 순환 연속성 — 이닝 첫 타자는 직전 이닝 마지막 타자의 다음 순번
- 투구수 단조증가 — $24 \rightarrow 38 \rightarrow 55 \rightarrow 63$ (이미 구현됨)
- 안타 수 합계 = 집계란 안타 수
- 타점 합계 = 득점 합계 (희생플라이 등 예외 처리 필요)

Claude Code에 줄 프롬프트 예시

```
stage4_verify.py에 잔루 정합 규칙을 추가해줘.

- 함수명 rule_lob_consistency, 기존 규칙들과 같은 시그니처
- 이닝 종료 시점에 reached != "none" 이고 scored == false 인 셀 수가
  집계란 잔루 값과 다르면 Violation(severity="error")
- verify()에서 호출하도록 연결
- tests/에 정상 케이스 1개, 위반 케이스 1개 pytest 추가
- 자동 보정은 만들지 마라. 검증만 한다.
```

부록 A — Claude Code 실전 사용법

PyCharm에서 자주 쓰는 조작

조작	용도
Alt+Ctrl+K	Claude Code 창 열기 (열린 파일이 맥락에 자동 포함)
에디터에서 선택 후 호출	선택 영역만 대상으로 질문·수정 요청
/clear	맥락 초기화. 주제가 바뀌면 자주 쓰세요
/undo	직전 파일 수정 되돌리기
Esc	작업 중단. 방향이 어긋나면 즉시 누르세요
@파일명	특정 파일을 맥락에 명시적으로 추가

좋은 프롬프트의 조건

이 프로젝트에서 특히 효과가 좋았던 패턴은 **증상 + 관찰 + 제약**입니다. '왜 안 되지?'가 아니라 '이 값이 이렇게 나왔는데, 이 조건을 지키면서 고쳐줘'로 쓰면 결과가 확연히 달라집니다.

나쁜 예	좋은 예
OCR이 잘 안 돼. 고쳐줘.	out_order 정확도가 0.72다. 틀린 셀 12개를 보니 II/III 혼동이 9개다. 프롬프트에서 로마숫자 지침을 강화하는 안을 2개 만들어줘.
stage 2 코드 짜줘.	stage2_slice.py에 셀별 잉크 비율 히스토그램을 뽑는 함수를 추가해줘. INK_THRESHOLD를 실측으로 정하는 게 목적이다.
테스트 만들어줘.	tests/golden/game001.json 대비 필드별 정확도를 내는 pytest를 만들어줘. 틀린 셀은 크롭 이미지 경로까지 출력할 것.

부록 B — 자주 빠지는 함정

1. 한 번에 다 만들려 함

"Stage 1부터 4까지 전부 구현해줘"라고 하면 그럴듯한 코드가 나오지만, 돌려보면 어디가 틀렸는지 알 수 없습니다. 한 단계씩, 눈으로 확인하며 가세요.

2. PROMPT_VERSION을 안 올림

프롬프트를 고쳤는데 결과가 그대로라면 십중팔구 이것입니다. CLAUDE.md에 규칙으로 적어두면 Claude Code가 알아서 챙겨줍니다.

3. 골든셋을 나중에 미룸

"일단 돌아가게 만들고 나중에 테스트"는 이 프로젝트에서 특히 위험합니다. VLM 출력은 그럴듯해 보이기 때문에, 정답과 대조하지 않으면 틀린 걸 알 수 없습니다.

4. 자동 보정을 일찍 켜

--auto-repair는 위반 패턴을 충분히 관찰하고 보정 규칙의 정밀도를 측정한 뒤에 하나씩 켜세요. 잘못된 보정은 검출 자체가 안 됩니다.

5. 에이전트로 만들고 싶어짐

매 시트가 같은 순서로 같은 108칸을 처리하므로 계획이 이미 정해져 있습니다. 에이전트는 재현성과 회귀 테스트를 잃는 대신 얻는 게 없습니다. 예외는 Stage 4의 재판독 루프 하나뿐이고, 그것도 위반 패턴을 본 뒤에 판단하세요.

다음 한 걸음

빈 양식 스캔본을 준비해 make_template.py probe를 돌리는 것부터 시작하세요. 좌표만 맞으면 나머지는 이미 깔린 배관 위에서 진행됩니다.